

Desarrollo Web Entorno Servidor

Unidad 6: Caso 1

Desarrollo de una aplicación web con acceso a bases de datos para gestionar información dinámicamente

Índice

1. [Introducción](#)
2. [Estructura del proyecto](#)
3. [Diseño de la base de datos](#)
4. [Conexión con la base de datos y acceso a datos](#)
5. [Consulta, recuperación y publicación de información](#)
6. [Actualización y eliminación de registros](#)
7. [Validación en frontend y backend](#)
8. [Pruebas realizadas](#)
9. [Mejoras añadidas](#)
10. [Conclusión](#)
11. [Referencias y recursos](#)

1. Introducción

En esta práctica se ha desarrollado una aplicación web en PHP conectada a una base de datos MySQL para la gestión de un catálogo de productos. El objetivo principal ha sido aplicar el acceso a bases de datos desde el entorno servidor, permitiendo recuperar, mostrar, insertar, actualizar y eliminar información almacenada de forma persistente.

El proyecto se ha planteado siguiendo una estructura organizada por capas, separando la presentación, la lógica de negocio y el acceso a datos. De este modo, la aplicación resulta más clara, mantenible y fácil de ampliar. A partir de esta base, se ha implementado un sistema CRUD completo sobre la tabla de productos, junto con validaciones en cliente y servidor para asegurar la consistencia de la información introducida.

Además de los requisitos principales de la tarea, se han incorporado algunas mejoras complementarias, como la gestión de imágenes asociadas a cada producto, filtros de visualización, ordenación por precio y soporte bilingüe opcional para parte del contenido. No obstante, el núcleo del trabajo se centra en la conexión con la base de datos, la ejecución de consultas SQL, la publicación de resultados en la interfaz y la modificación segura de los registros.

2. Estructura del proyecto

La aplicación se ha organizado siguiendo una separación por capas, con el fin de distinguir con claridad la interfaz, la lógica de negocio y el acceso a los datos. Esta estructura facilita el mantenimiento del código y evita mezclar en un mismo archivo responsabilidades diferentes.

Por un lado, la capa de presentación se encarga de mostrar la información al usuario y de recoger sus acciones mediante formularios, botones y elementos visuales de la interfaz. En esta parte se encuentran las páginas principales de la aplicación, los estilos CSS y los scripts de cliente que gestionan la interacción y parte de la validación inicial.

Por otro lado, la lógica de negocio actúa como intermediaria entre la interfaz y el acceso a datos. Su función es recibir las peticiones de la aplicación, comprobar que la información tenga sentido y decidir qué operación debe realizarse en cada caso, como listar productos, crear un nuevo registro, modificar uno existente o eliminarlo.

Finalmente, la capa de acceso a datos concentra la conexión con la base de datos y la ejecución de consultas SQL. Gracias a esta separación, las operaciones sobre MySQL quedan centralizadas en archivos específicos, lo que permite reutilizar código, trabajar con sentencias preparadas y reducir errores.

Esta organización aporta mayor claridad al desarrollo del proyecto y permite que cada parte de la aplicación cumpla una función concreta dentro del sistema general.

 datos	08/03/2026 20:51	File folder
 IMG	08/03/2026 22:18	File folder
 negocio	08/03/2026 22:16	File folder
 sql	08/03/2026 22:03	File folder
 tests	08/03/2026 22:44	File folder
 carrito	08/03/2026 21:17	PHP Source File
 i18n	08/03/2026 23:10	JavaScript Source ...
 index	08/03/2026 22:21	PHP Source File
 noticias	08/03/2026 22:21	PHP Source File
 preferencias	08/03/2026 22:58	JavaScript Source ...
 script	08/03/2026 23:10	JavaScript Source ...
 sobre	08/03/2026 22:21	PHP Source File
 style	08/03/2026 22:14	CSS Source File
 tienda	08/03/2026 23:10	PHP Source File

Figura 1. Estructura general de carpetas y archivos del proyecto.

3. Diseño de la base de datos

Para el desarrollo de la aplicación se ha creado una base de datos en MySQL llamada *tienda_ud6*, orientada al almacenamiento de un catálogo de productos. Dentro de esta base de datos se ha definido una tabla principal, *productos*, que reúne la información necesaria para mostrar, añadir, modificar y eliminar registros desde la aplicación web.

La estructura de la tabla se ha diseñado para cubrir los datos básicos de cada producto. Entre los campos principales se encuentran el identificador único, el nombre, la descripción, la categoría, el precio, el nombre del archivo de imagen y la fecha de creación del registro. De esta forma, cada producto puede almacenarse de manera individual y recuperarse posteriormente para su publicación en la interfaz.

El campo *id* se ha configurado como clave primaria autoincremental, lo que permite identificar cada registro de forma única sin necesidad de asignar manualmente su valor. Los campos *nombre*, *descripcion*, *categoria* y *precio* se han marcado como obligatorios, ya que constituyen la información esencial del producto. Por su parte, el campo de imagen se utiliza para asociar cada registro con un archivo gráfico

almacenado en el proyecto, mientras que la fecha de creación permite conservar una referencia temporal del momento en que se registró cada elemento.

Como parte de la versión final del proyecto, la tabla también incorpora varios campos opcionales en inglés, destinados a almacenar traducciones del nombre, la descripción y la categoría de cada producto. Esta ampliación no era imprescindible para cumplir los requisitos mínimos de la práctica, pero se añadió como mejora complementaria para enriquecer la presentación del catálogo y ampliar sus posibilidades de uso.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 nombre	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 nombre_en	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 descripcion	text	utf8mb4_general_ci		No	None			Change Drop More
☐	5 descripcion_en	text	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	6 categoria	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	7 categoria_en	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	8 precio	decimal(10,2)			No	None			Change Drop More
☐	9 imagen	varchar(255)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	10 creado_en	timestamp			No	current_timestamp()			Change Drop More

Figura 2. Estructura final de la tabla productos en la base de datos tienda_ud6.

	id	nombre	nombre_en	descripcion	descripcion_en	categoria	categoria_en	precio	imagen	creado_en
☐	1	Teclado mecánico	Mechanical keyboard	Teclado con diseño mecánico orientado a escritura...	Mechanical design keyboard focused on typing and g...	Periféricos	Peripherals	59.90	teclado.jpg	2026-03-08 20:19:22
☐	2	Ratón inalámbrico	Wireless mouse	Ratón inalámbrico ergonómico para uso diario	Ergonomic wireless mouse for daily use	Periféricos	Peripherals	24.50	raton.jpg	2026-03-08 20:19:22
☐	3	Monitor 24"	24" monitor	Monitor de 24 pulgadas adecuado para trabajo y con...	24-inch monitor suitable for work and multimedia c...	Pantallas	Displays	139.00	monitor.jpg	2026-03-08 20:19:22
☐	4	USB-C Hub	USB-C hub	Adaptador multipuerto USB-C para ampliar conexione...	Multiport USB-C adapter to expand connectivity	Accesorios	Accessories	29.95	hub.jpg	2026-03-08 20:19:22
☐	5	Monitor 27"	27" monitor	Monitor de 27 pulgadas con resolución Full HD para...	27-inch Full HD monitor for work and entertainment	Pantallas	Displays	189.00	monitor27.jpg	2026-03-08 20:19:49
☐	6	Soporte para portátil	Laptop stand	Soporte ajustable para mejorar la ergonomía del po...	Adjustable stand to improve laptop ergonomics	Accesorios	Accessories	34.90	soporte.jpg	2026-03-08 20:19:49
☐	7	Auriculares inalámbricos	Wireless headphones	Auriculares Bluetooth con sonido estéreo y microfó...	Bluetooth headphones with stereo sound and integra...	Audio	Audio	79.95	auriculares.jpg	2026-03-08 20:19:49
☐	8	Altavoces compactos	Compact speakers	Pareja de altavoces compactos para escritorio	Pair of compact desktop speakers	Audio	Audio	45.50	altavoces.jpg	2026-03-08 20:19:49

Figura 3. Registros de ejemplo almacenados en la tabla productos.

El diseño de la base de datos responde a una necesidad práctica y realista, manteniendo una estructura sencilla, clara y suficiente para soportar las operaciones CRUD requeridas en la práctica.

4. Conexión con la base de datos y acceso a datos

La conexión entre la aplicación y la base de datos se realiza mediante PDO, una extensión de PHP que permite trabajar con distintos sistemas gestores de bases de datos a través de una interfaz común. En este proyecto se ha utilizado para establecer la comunicación con MySQL de forma segura y estructurada.

Para centralizar esta tarea, se creó un archivo específico de conexión en la capa de datos. En él se definen los parámetros necesarios, como el servidor, el nombre de la base de datos, el usuario y la codificación de caracteres. Además, se configuró el modo de error para que PDO lance excepciones, lo que facilita la detección y el control de posibles fallos durante la ejecución.

```

<?php
/**
 * Conexión PDO centralizada para el proyecto.
 * Usa MySQL local de XAMPP y la base de datos ya creada por el usuario.
 */

$host = '127.0.0.1';
$dbname = 'tienda_ud6';
$user = 'root';
$pass = '';

$dsn = "mysql:host={$host};dbname={$dbname};charset=utf8mb4";
$options = [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
];

try {
    $conexion = new PDO($dsn, $user, $pass, $options);
} catch (PDOException $e) {
    http_response_code(500);
    die('Error de conexión a la base de datos: ' . $e->getMessage());
}

```

Código 1. Fragmento del archivo de conexión a la base de datos mediante PDO.

A partir de esta conexión, las operaciones de acceso a datos se concentran en el repositorio de la aplicación. Este componente es el encargado de ejecutar las consultas SQL necesarias para recuperar, insertar, actualizar o eliminar productos. De este modo, la lógica de acceso a la base de datos no queda repartida por distintos archivos, sino agrupada en una parte concreta del proyecto, lo que mejora la organización general del código.

Otro aspecto importante es el uso de sentencias preparadas. Este mecanismo permite enviar los valores de forma separada respecto a la consulta SQL, lo que reduce riesgos de inyección SQL y aporta mayor seguridad al tratamiento de los datos recibidos desde formularios o peticiones de la aplicación.

La solución adoptada permite mantener una conexión estable con la base de datos y concentrar el acceso a la información en una parte concreta del proyecto. Gracias a ello, el código resulta más ordenado, más fácil de mantener y más seguro frente a errores o usos indebidos de los datos recibidos.

5. Consulta, recuperación y publicación de información

Una de las funciones principales de la aplicación consiste en consultar la información almacenada en la base de datos y publicarla de forma comprensible en la interfaz. Para ello, el sistema recupera los registros de la tabla *productos* mediante consultas SQL y los transforma en una salida visual organizada dentro del catálogo.

La obtención de los datos se realiza desde la capa de acceso, donde se ejecutan las consultas necesarias para listar todos los productos o aplicar determinadas condiciones de búsqueda y filtrado. Entre estas operaciones se incluyen la recuperación completa del catálogo, la búsqueda por texto y el filtrado por categoría, además de la ordenación por precio en función del criterio seleccionado por el usuario.

```
/**
 * Devuelve todos los productos o los productos filtrados por término.
 * Los campos para el frontend se normalizan a: id, nombre, descripcion, categoria,
 * precio, imagen.
 */
public function buscar(string $term = '', string $orden = 'desc', string $categoria
= '', string $idioma = 'es'): array {
    // Normalización de entrada para evitar espacios sobrantes en filtros.
    $term = trim($term);
    $categoria = trim($categoria);
    $ordenSql = strtolower($orden) === 'asc' ? 'ASC' : 'DESC';
    $usarIngles = $this->soportaIngles && strtolower($idioma) === 'en';

    // Selección dinámica de columnas (ES por defecto, EN con respaldo si existe
    traducción).
    $campoNombre = $usarIngles ? "IFNULL(NULLIF(nombre_en, ''), nombre)" : 'nombre';
    $campoDescripcion = $usarIngles ? "IFNULL(NULLIF(descripcion_en, ''),
    descripcion)" : 'descripcion';
    $campoCategoria = $usarIngles ? "IFNULL(NULLIF(categoria_en, ''), categoria)" :
    'categoria';

    $where = [];
    $params = [];

    if ($term !== '') {
        // Búsqueda por coincidencia parcial en nombre, categoría o descripción.
        $where[] = "({$campoNombre} LIKE :termNombre OR {$campoCategoria} LIKE
        :termCategoria OR {$campoDescripcion} LIKE :termDescripcion)";
        $like = '%' . $term . '%';
        $params[':termNombre'] = $like;
        $params[':termCategoria'] = $like;
        $params[':termDescripcion'] = $like;
    }

    if ($categoria !== '') {
        // Filtro exacto de categoría cuando el usuario selecciona una opción
```

```

concreta.
    $where[] = "{$campoCategoria} = :categoriaFiltro";
    $params[':categoriaFiltro'] = $categoria;
}

$sql = "SELECT
        id,
        {$campoNombre} AS nombre,
        {$campoDescripcion} AS descripcion,
        {$campoCategoria} AS categoria,
        precio,
        imagen,
        creado_en
    FROM productos
    ORDER BY precio {$ordenSql}";

if (!empty($where)) {
    // Inserta condiciones WHERE antes del ORDER BY para mantener una única
    // plantilla SQL.
    $sql = str_replace('ORDER BY', 'WHERE ' . implode(' AND ', $where) . ' ORDER
    BY', $sql);
}

$stmt = $this->db->prepare($sql);
foreach ($params as $key => $value) {
    $stmt->bindValue($key, $value, PDO::PARAM_STR);
}
$stmt->execute();
return $stmt->fetchAll();
}

```

Código 2. Método utilizado para consultar, filtrar y recuperar productos desde la base de datos.

Una vez obtenida la información, la aplicación la muestra en la interfaz principal mediante tarjetas o bloques visuales que recogen los datos más relevantes de cada producto. De este modo, el usuario puede identificar con rapidez el nombre, la descripción, la categoría, el precio y la imagen asociada a cada registro. Esta publicación de resultados permite comprobar de forma directa que los datos almacenados en MySQL se recuperan correctamente y se integran en la parte visible de la aplicación.

Además, el proyecto incorpora una mejora complementaria relacionada con la visualización bilingüe. Cuando existen datos opcionales en inglés, la interfaz puede utilizarlos para mostrar parte del contenido en ese idioma. En caso contrario, se mantiene la información principal almacenada en español, evitando que la ausencia de traducción impida la publicación del producto.

Esta parte del desarrollo resulta esencial, ya que demuestra que la base de datos no solo almacena información, sino que también alimenta dinámicamente la interfaz de la aplicación, permitiendo trabajar con contenidos reales y actualizados.

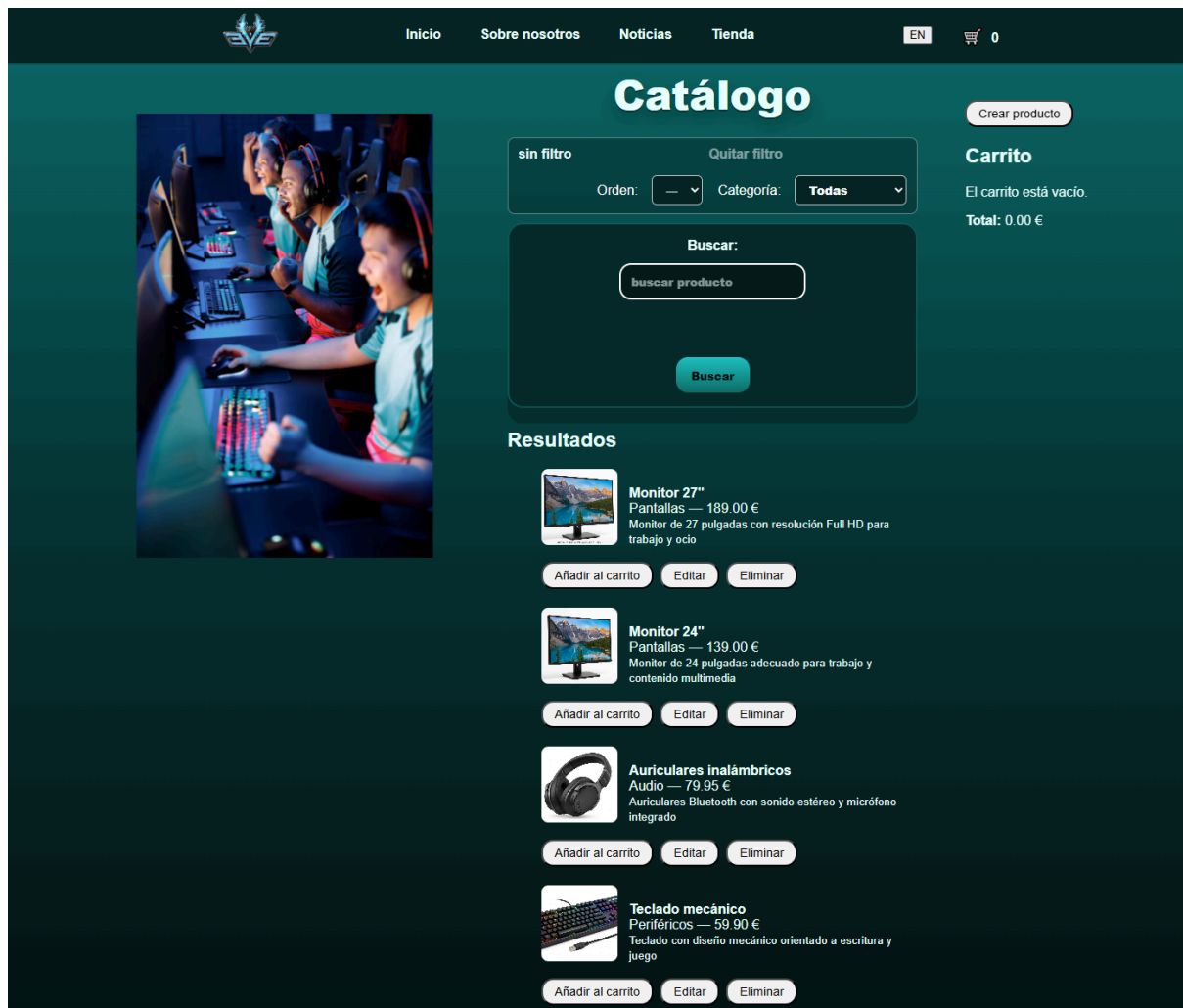


Figura 4. Publicación de los productos recuperados desde la base de datos en la interfaz principal de la aplicación.

6. Actualización y eliminación de registros

Además de consultar y mostrar la información almacenada, la aplicación permite modificar y eliminar productos ya existentes en la base de datos. Estas dos operaciones completan el ciclo básico de gestión de registros y resultan fundamentales dentro del modelo CRUD desarrollado en la práctica.

La actualización de productos se realiza a partir de un formulario de edición que recupera previamente los datos del registro seleccionado. De esta forma, el usuario puede visualizar la información actual del producto, modificar los campos necesarios y enviar los cambios para que se reflejen en la base de datos. Este proceso evita tener que crear un nuevo registro desde cero cuando solo se necesita corregir o actualizar parte de la información.

Por otro lado, la eliminación permite borrar de forma directa un producto almacenado. Esta operación se ejecuta sobre el identificador del registro correspondiente, garantizando que solo se elimine el elemento seleccionado.

Gracias a ello, la aplicación puede mantener el catálogo actualizado y eliminar productos que ya no deban mostrarse en la interfaz.

```
public function actualizar(int $id, array $data): bool {
    // Actualiza producto existente por ID, incluyendo traducciones cuando están disponibles.
    if ($this->soportaIngles) {
        $setSql = [
            'nombre = :nombre',
            'descripcion = :descripcion',
            'categoria = :categoria',
            'precio = :precio',
            'imagen = :imagen',
        ];
        if (array_key_exists('nombre_en', $data)) {
            $setSql[] = 'nombre_en = :nombre_en';
        }
        if (array_key_exists('descripcion_en', $data)) {
            $setSql[] = 'descripcion_en = :descripcion_en';
        }
        if (array_key_exists('categoria_en', $data)) {
            $setSql[] = 'categoria_en = :categoria_en';
        }
        $sql = "UPDATE productos SET " . implode(', ', $setSql) . " WHERE id = :id";
    } else {
        $sql = "UPDATE productos
            SET nombre = :nombre,
                descripcion = :descripcion,
                categoria = :categoria,
                precio = :precio,
                imagen = :imagen
            WHERE id = :id";
    }

    $stmt = $this->db->prepare($sql);
    $stmt->bindValue(':id', $id, PDO::PARAM_INT);
    $stmt->bindValue(':nombre', $data['nombre']);
    $stmt->bindValue(':descripcion', $data['descripcion']);
    $stmt->bindValue(':categoria', $data['categoria']);
    $stmt->bindValue(':precio', $data['precio']);
    if (($data['imagen'] ?? null) === null || $data['imagen'] === '') {
        // Permite limpiar la imagen (valor NULL) si el usuario deja el campo vacío.
        $stmt->bindValue(':imagen', null, PDO::PARAM_NULL);
    } else {
        $stmt->bindValue(':imagen', $data['imagen'], PDO::PARAM_STR);
    }

    if ($this->soportaIngles) {
        if (array_key_exists('nombre_en', $data)) {
            $stmt->bindValue(':nombre_en', $data['nombre_en']);
        }
        if (array_key_exists('descripcion_en', $data)) {
            $stmt->bindValue(':descripcion_en', $data['descripcion_en']);
        }
        if (array_key_exists('categoria_en', $data)) {
            $stmt->bindValue(':categoria_en', $data['categoria_en']);
        }
    }

    return $stmt->execute();
}
```

Código 3. Fragmento del método utilizado para actualizar los datos de un producto en la base de datos.

Tanto la actualización como la eliminación se gestionan desde la lógica del servidor y se apoyan en consultas SQL específicas, ejecutadas a través del repositorio de acceso a datos. Esto permite mantener separadas la interfaz y las operaciones sobre MySQL, además de centralizar el tratamiento de los cambios realizados sobre la información.

La incorporación de estas funciones demuestra que la aplicación no se limita a consultar datos, sino que también permite intervenir sobre ellos de manera controlada, manteniendo la persistencia y la coherencia de los registros almacenados.



Figura 5. Producto mostrado en el catálogo con las opciones disponibles para editar o eliminar el registro.

Nombre (base ES)

Monitor 27"

Descripción (base ES)

Monitor de 27 pulgadas con resolución Full HD para trabajo y ocio

Categoría (base ES)

Pantallas

Precio

189.00

Archivo de imagen

monitor27.jpg

Subir imagen

Choose File

No file chosen

Nombre (inglés opcional)

Descripción (inglés opcional)

Categoría (inglés opcional)

Guardar cambios

Cancelar

Figura 6. Formulario de edición con los datos del producto cargados para su modificación.

7. Validación en frontend y backend

La aplicación incorpora validación de datos tanto en el lado del cliente como en el servidor. Esta doble comprobación resulta necesaria para evitar la introducción de información incorrecta y para reforzar la seguridad del sistema, ya que la validación realizada únicamente en el navegador puede ser modificada o incluso omitida.

En el frontend, la validación se aplica sobre los formularios de alta y edición de productos. Antes de enviar la información, se comprueba que los campos obligatorios estén cumplimentados y que los valores introducidos respeten unas condiciones mínimas de formato y coherencia. De este modo, se mejora la experiencia del usuario al detectar errores de forma inmediata, sin necesidad de esperar a que el servidor procese la petición.

```
// Validación frontend previa a crear/actualizar producto.
function validateFormData(data) {
  if (!data.nombre || data.nombre.trim().length < 2) return t('val_nombre', 'El nombre debe tener al menos 2 caracteres.', 'Name must have at least 2 characters.');
```

```
  if (!data.descripcion || data.descripcion.trim().length < 5) return t('val_descripcion', 'La descripción debe tener al menos 5 caracteres.', 'Description must have at least 5 characters.');
```

```
  if (!data.categoria || data.categoria.trim().length < 1) return t('val_categoria', 'La categoría es obligatoria.', 'Category is required.');
```

```
  if (Number.isNaN(Number(data.precio)) || Number(data.precio) < 0) return t('val_precio', 'El precio debe ser un número mayor o igual que 0.', 'Price must be a number greater than or equal to 0.');
```

```
  if (data.imagen && data.imagen.trim() !== '' && !/^[a-zA-Z0-9._-]+$/.test(data.imagen.trim())) return t('val_imagen', 'La imagen debe ser un nombre de archivo válido (ejemplo: teclado.jpg).', 'Image must be a valid filename (example: keyboard.jpg).');
```

```
  return null;
}
```

Código 4. Fragmento de validación en cliente aplicado a los formularios de producto.

En el backend, la validación se realiza de nuevo antes de ejecutar cualquier operación sobre la base de datos. Esta segunda capa de control verifica que los datos recibidos sean válidos y adecuados para su almacenamiento, evitando que se inserten registros incompletos, inconsistentes o manipulados. Además, esta comprobación del lado del servidor actúa como medida de seguridad frente a envíos alterados manualmente o peticiones construidas fuera de la interfaz habitual de la aplicación.

```
function validateProductData(array $data): array {
  // Validación de negocio y sanitización básica de campos de producto.
  $nombre = trim((string)($data['nombre'] ?? ''));
  $descripcion = trim((string)($data['descripcion'] ?? ''));
  $categoria = trim((string)($data['categoria'] ?? ''));
  $precioRaw = $data['precio'] ?? '';
  $imagen = trim((string)($data['imagen'] ?? ''));

  if ($nombre === '' || mb_strlen($nombre) < 2 || mb_strlen($nombre) > 100) {
    return ['ok' => false, 'message' => 'Nombre inválido (2-100 caracteres).'];
  }

  if ($descripcion === '' || mb_strlen($descripcion) < 5) {
    return ['ok' => false, 'message' => 'Descripción inválida (mínimo 5 caracteres).'];
  }
}
```

```

    if ($categoria === '' || mb_strlen($categoria) > 100) {
        return ['ok' => false, 'message' => 'Categoría inválida (1-100
caracteres).'];
    }
    if (!is_numeric($precioRaw) || (float)$precioRaw < 0) {
        return ['ok' => false, 'message' => 'Precio inválido (número >= 0).'];
    }
    if ($imagen !== '' && !preg_match('/^[a-zA-Z0-9._-]+$/', $imagen)) {
        return ['ok' => false, 'message' => 'Imagen inválida (solo nombre de
archivo).'];
    }

    $cleanData = [
        'nombre' => $nombre,
        'descripcion' => $descripcion,
        'categoria' => $categoria,
        'precio' => number_format((float)$precioRaw, 2, '.', ''),
        'imagen' => $imagen === '' ? null : $imagen,
    ];

    if (array_key_exists('nombre_en', $data)) {
        $cleanData['nombre_en'] = trim((string)$data['nombre_en']);
    }
    if (array_key_exists('descripcion_en', $data)) {
        $cleanData['descripcion_en'] = trim((string)$data['descripcion_en']);
    }
    if (array_key_exists('categoria_en', $data)) {
        $cleanData['categoria_en'] = trim((string)$data['categoria_en']);
    }

    return [
        'ok' => true,
        'data' => $cleanData,
    ];
}

```

Código 5. Fragmento de validación en servidor previo a la operación sobre la base de datos.

La validación en ambos niveles permite mantener una mayor coherencia en la información almacenada y refuerza la fiabilidad de las operaciones CRUD implementadas. Gracias a ello, la aplicación no solo interactúa con la base de datos, sino que también controla la calidad de los datos que se incorporan o modifican en ella.

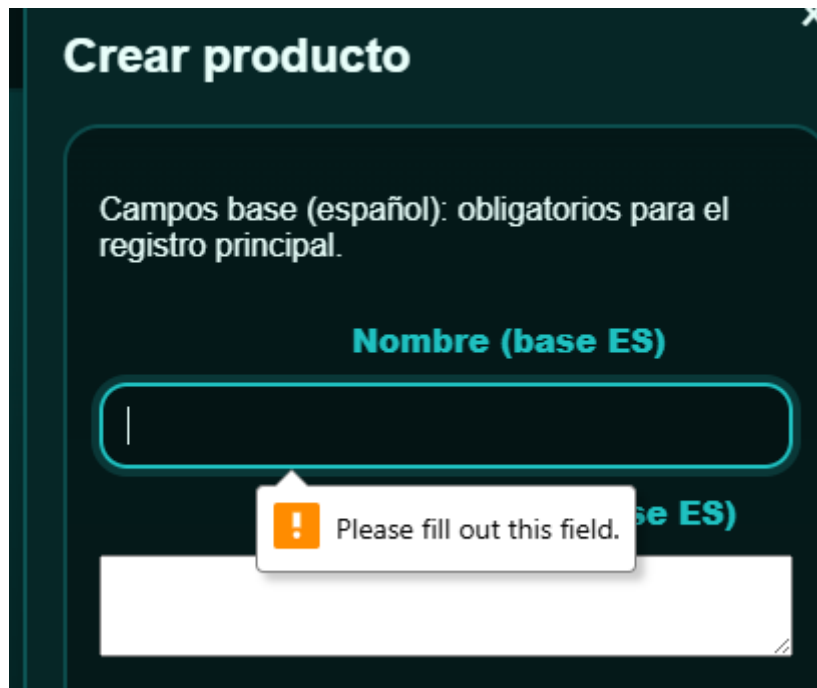


Figura 7. Ejemplo de validación en cliente durante la introducción de datos en el formulario.

8. Pruebas realizadas

Una vez implementadas las funcionalidades principales de la aplicación, se realizaron distintas pruebas para comprobar que la conexión con la base de datos, las consultas SQL y las operaciones CRUD funcionaban correctamente. Estas comprobaciones permitieron verificar no solo el acceso a MySQL, sino también la correcta integración entre la capa de datos, la lógica de negocio y la interfaz visible para el usuario.

En primer lugar, se comprobó la recuperación de productos desde la base de datos y su publicación en la interfaz principal. También se verificó el funcionamiento de la búsqueda por texto, el filtrado por categoría y la ordenación por precio, asegurando que los resultados mostrados coincidieran con los registros almacenados en la tabla *productos*.

A continuación, se probaron las operaciones de alta, edición y eliminación. En cada caso, se revisó que los cambios realizados se reflejaran correctamente tanto en la interfaz como en la base de datos. Estas pruebas permitieron confirmar que la aplicación no solo consulta información, sino que también mantiene la persistencia de los datos tras insertar, modificar o borrar registros.

Por último, se revisó la validación de formularios en cliente y en servidor, comprobando que los campos obligatorios no admitieran valores vacíos y que los datos numéricos y textuales respetaran las restricciones definidas. De esta manera, se validó que la aplicación responde de forma adecuada ante entradas erróneas y que protege la base de datos frente a valores no válidos.

```

d6_proyecto > tests > {} informe_pruebas.json > ...
1  {
2    "generatedAt": "2026-03-08T23:13:00.5831467+01:00",
3    "baseUrl": "http://localhost/dwes/ud6_caso1/ud6_proyecto",
4    "total": 6,
5    "passed": 6,
6    "failed": 0,
7    "tests": [
8      {
9        "name": "GET listado productos",
10       "passed": true,
11       "details": "Productos obtenidos: 8",
12       "timestamp": "2026-03-08T23:13:00.3978649+01:00"
13     },
14     {
15       "name": "GET categorias",
16       "passed": true,
17       "details": "Categorias obtenidas: Accessories, Audio, Displays, Peripherals",
18       "timestamp": "2026-03-08T23:13:00.4347131+01:00"
19     },
20     {
21       "name": "POST crear producto",
22       "passed": true,
23       "details": "ID creado: 18",
24       "timestamp": "2026-03-08T23:13:00.4878527+01:00"
25     },
26     {
27       "name": "POST actualizar producto",
28       "passed": true,
29       "details": "ID actualizado: 18",
30       "timestamp": "2026-03-08T23:13:00.5308658+01:00"
31     },
32     {
33       "name": "POST eliminar producto",
34       "passed": true,
35       "details": "ID eliminado: 18",
36       "timestamp": "2026-03-08T23:13:00.5609195+01:00"
37     },
38     {
39       "name": "GET subida protegida",
40       "passed": true,
41       "details": "Devuelve 405 correctamente.",
42       "timestamp": "2026-03-08T23:13:00.5756341+01:00"
43     }
44   ]
45 }
46

```

Figura 8. Resultado de las comprobaciones realizadas sobre las funciones principales de la aplicación.

Las pruebas realizadas permitieron confirmar el correcto funcionamiento general del proyecto. En particular, se verificó que las funciones principales de consulta, inserción, actualización, eliminación, búsqueda y filtrado respondían correctamente, lo que permitió dar por cubiertos los objetivos principales de la práctica.

9. Mejoras añadidas

Además de los requisitos principales de la práctica, el proyecto incorpora varias funcionalidades complementarias que amplían su utilidad y mejoran la experiencia de uso. Estas mejoras no eran imprescindibles para cumplir el objetivo básico de acceso a bases de datos y operaciones CRUD, pero se integraron para enriquecer el resultado final y dotar a la aplicación de un comportamiento más completo.

Una de las mejoras incorporadas ha sido la gestión de imágenes asociadas a cada producto. Cada registro puede mostrar un archivo gráfico almacenado en el proyecto, lo que permite presentar el catálogo de una forma más visual y facilita la identificación de los elementos publicados. Esta característica aporta un resultado más cercano a una tienda real y mejora la presentación de la información recuperada desde la base de datos.

También se añadieron funciones de búsqueda, filtrado por categoría y ordenación por precio. Estas opciones permiten trabajar con el catálogo de una forma más cómoda y dinámica, ya que el usuario puede localizar productos concretos, limitar los resultados a una categoría determinada o reorganizar la información según el criterio seleccionado. Aunque no eran el núcleo de la tarea, estas operaciones complementan de manera natural la consulta de datos y refuerzan la utilidad práctica de la aplicación.

Otra mejora destacable es la incorporación de soporte bilingüe opcional para determinados campos del producto. En la versión final del proyecto, además de los campos principales en español, la base de datos permite almacenar traducciones opcionales en inglés para el nombre, la descripción y la categoría. Para evitar confusiones, los formularios de creación y edición diferencian de forma explícita los campos base obligatorios en español y los campos opcionales de traducción. De esta forma, la ampliación bilingüe se integra como una mejora adicional sin interferir en el funcionamiento principal del sistema.

Asimismo, el proyecto incorpora una subida controlada de imágenes y un carrito visual. La primera permite asociar archivos gráficos a los productos desde el formulario correspondiente, mientras que el segundo añade una pequeña funcionalidad de interacción que permite seleccionar productos y mostrarlos de forma acumulada en la interfaz. Aunque estas características no constituyen el eje principal de la práctica, sí contribuyen a presentar una aplicación más completa y mejor conectada con un contexto de uso real.

Estas mejoras amplían el proyecto más allá del mínimo exigido, pero siempre manteniendo como base la conexión con MySQL, la gestión de productos y la validación de datos como elementos principales del desarrollo.



Figura 11. Carrito visual actualizado tras añadir productos desde el catálogo.



Figura 12. Opciones de filtrado por categoría y ordenación por precio añadidas para facilitar la consulta del catálogo.

10. Conclusión

A lo largo de esta práctica se ha desarrollado una aplicación web en PHP conectada a una base de datos MySQL, orientada a la gestión de un catálogo de productos. El proyecto ha permitido trabajar de forma práctica con la conexión a bases de datos desde el entorno servidor, la ejecución de consultas SQL y la implementación completa de operaciones CRUD sobre registros reales.

Además de la recuperación y publicación de información en la interfaz, se han incorporado funciones de inserción, actualización y eliminación, junto con validaciones tanto en cliente como en servidor para reforzar la fiabilidad del sistema. Esto ha permitido comprobar no solo el acceso a los datos, sino también su tratamiento controlado y su persistencia dentro de la aplicación.

El desarrollo se ha organizado mediante una estructura separada por capas, diferenciando la presentación, la lógica de negocio y el acceso a datos. Esta organización ha facilitado la comprensión del proyecto y ha contribuido a mantener una distribución más clara de responsabilidades entre los distintos archivos.

Junto a los requisitos principales, también se añadieron mejoras complementarias, como la gestión de imágenes, el filtrado y la ordenación del catálogo, el soporte bilingüe opcional y un pequeño carrito visual. Estas ampliaciones han servido para enriquecer la aplicación sin desviar el objetivo central de la práctica, que era

demostrar el trabajo con bases de datos y operaciones sobre los registros almacenados.

En definitiva, la práctica ha permitido aplicar de forma realista varios de los contenidos trabajados en la unidad, dando como resultado una aplicación funcional, estructurada y capaz de interactuar con la base de datos de forma segura y dinámica.

11. Referencias y recursos

Para la elaboración del proyecto se han utilizado distintos recursos de apoyo:

- *Mimo App y Enki App*
- *Material de la unidad didáctica*
- *Base de código del trabajo desarrollado en la unidad 5.*
- *Google Gemini: Recurso utilizado para generación de imágenes de los productos.*
- *SQL Squid Game: app web para practicar SQL jugando*
<https://datalemur.com/sql-game>
- *CodePen*
<https://codepen.io/pen>
- *Program with Gio: Learn PHP The Right Way - Full PHP Tutorial for Beginners & Advanced (Youtube)*
https://youtube.com/playlist?list=PLr3d3QYzkw2xabQRUpCZ_IBk9W50M9pe-&si=Lt00-XbHoDhLyy8i
- *MDN Web Docs. Validación de formularios*
https://developer.mozilla.org/es/docs/Learn/Forms/Form_validation
- *phpMyAdmin*
<https://www.phpmyadmin.net/>